

L10c: Stochastic Multi-Armed Bandits

Exploration, regret, UCB1, and Thompson sampling

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How should an agent choose among actions with unknown rewards, when it sees only the reward of the action it takes?

Today's concepts

- Balance exploration and exploitation: choose between trying actions to learn about them and taking the action that looks best so far, and see why an agent needs both.
- Solve multi-armed bandit problems: choose the next arm with the explore-first, epsilon-greedy, UCB1, and Thompson sampling algorithms.
- Compare bandit algorithms with regret: define the regret of a bandit algorithm and compare the expected-regret bounds of explore-first, epsilon-greedy, and UCB1.

Lecture: Stochastic Multi-Armed Bandit Problems

Exploration vs. Exploitation

An agent chooses an action, receives a reward, and uses that observation to decide what to try next. Its goal is to collect as much reward as possible.

- Exploration: try actions that have not been tried often, to learn about their rewards.
- Exploitation: take the action that looks best so far. An agent that only exploits never tries anything new; how boring!

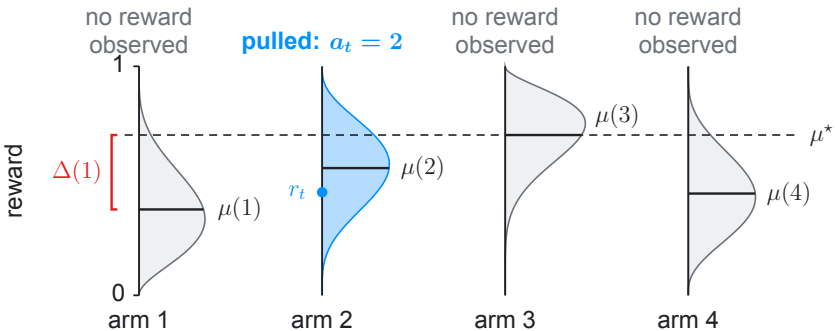
Explore too much, and we give up reward now. Exploit too much, and we may write off an action whose low observed reward was simply bad luck.

Discuss

Think of a choice you make again and again, such as where to eat lunch. How do you decide when to try something new and when to go back to your favorite?

What Is a Bandit Problem?

An agent repeatedly chooses among K options, called arms, and receives a reward from the arm it pulls.



Only the pulled arm reports a reward. In round t, arm a_t yields reward r_t; $\mu^\star$ is the largest of the unseen means $\mu(a)$, and $\Delta(1)$ is arm 1's gap below it.

Bandit Problems in Practice

Each system tries one option at a time and sees only that option's outcome:

Application	Arms	Reward of one pull	Cost of exploring
Clinical trials Villar et al., 2015	treatments given to patients	1 if the treatment works, 0 if not	some patients get a less effective treatment
News recommendation Li et al., 2010	articles for the Yahoo! front page	1 if the visitor clicks, 0 if not	some visitors see a less popular article
Network routing Talebi et al., 2018	paths through a computer network	time for the data to arrive	some data take a slower path
Online pricing Misra et al., 2019	candidate prices for a product	profit on the sale, 0 if no sale	some customers see a less profitable price
Reaction conditions Wang et al., 2024	catalyst, solvent, and base choices	yield of the reaction, from 0 to 1	some runs use poor conditions

Delay and profit are rescaled to $[0, 1]$, with faster or more profitable closer to 1.

Stochastic Bandit Model

The agent has a fixed set of arms $\mathcal{A}$, with $|\mathcal{A}| = K$, and plays for T rounds. Arm a draws its rewards from a distribution $\mathcal{D}_a$ on $[0, 1]$, with mean reward $\mu(a) = \mathbb{E}_{r \sim \mathcal{D}_a}[r]$.

For $t = 1, 2, \ldots, T$ **do**:

1. The agent chooses an arm $a_t \in \mathcal{A}$ using its previous actions and observed rewards.
2. The environment independently draws $r_t \sim \mathcal{D}_{a_t}$.
3. The agent receives r_t and updates its estimates.

The regret bounds assume independent draws, stationary distributions (they do not change), and rewards between 0 and 1. Maximizing total reward is the same goal as minimizing regret.

Regret

Let $\mu^\star = \max_{a \in \mathcal{A}} \mu(a)$. Always pulling the best arm earns $T\mu^\star$ in expectation, so the regret of one game (one run of T rounds) and the cumulative gap loss are:

$$R(T) = T\mu^\star - \sum_{t=1}^{T} r_t, \qquad G(T) = \sum_{t=1}^{T} \Delta(a_t), \quad \Delta(a) = \mu^\star - \mu(a).$$

- $R(T)$ moves with reward luck, even after the agent has found the best arm.
- $G(T)$ adds the mean reward given up by each pull, so a pull of the best arm adds nothing.

A pull of a_t has expected reward $\mu(a_t)$, so the two agree on average:

$$\boxed{\mathbb{E}[R(T)] = \mathbb{E}[G(T)]}$$

Explore-First Algorithm

Pull each arm N times (exploration), then pull the arm with the highest sample mean for the rest of the game (exploitation).

Initialize: Given K arms and the number of rounds $T \geq K$, choose

$$N = \min\left\{\left\lfloor \frac{T}{K} \right\rfloor,\ \max\left\{1,\ \text{round}\left[\left(\frac{T}{K}\right)^{2/3} (\log T)^{1/3}\right]\right\}\right\}.$$

1. For each arm $a \in \mathcal{A}$ **do**: pull arm a for N rounds and compute its sample mean $\bar{\mu}(a)$, the average of its N rewards.
2. Choose $\hat{a} \leftarrow \arg\max_{a \in \mathcal{A}} \bar{\mu}(a)$, breaking ties by the first arm index.
3. For $t = KN + 1, \ldots, T$ **do**: pull arm $\hat{a}$.

Here, $\log$ is the natural logarithm; the $\max$ pulls every arm at least once, and the $\min$ fits the exploration in T rounds.

Explore-First: Regret Bound

With N of order $(T/K)^{2/3}(\log T)^{1/3}$, explore-first achieves (Slivkins, Theorem 1.5):

$$\boxed{\mathbb{E}[R(T)] \leq T^{2/3} \cdot \mathcal{O}(K \log T)^{1/3}}$$

This N balances the cost of exploring against the cost of committing to a poorly estimated arm.

- Of order x: a constant multiple of x.
- Big-O, $\mathcal{O}(x)$: at most a constant multiple of x.

Neither constant depends on T or K.

Epsilon-Greedy Algorithm

Spread exploration out: pull a random arm with probability ϵ_t, which shrinks over time, and the best arm so far otherwise.

Initialize: Given K arms and $T \geq K$ rounds, pull each arm once and call its reward r_a; set its pull count $N_a \leftarrow 1$ and sample mean $\bar{\mu}(a) \leftarrow r_a$.

For $t = K+1, \ldots, T$ **do**:

1. Draw u uniformly from $[0, 1]$ and compute $\epsilon_t = \min\left\{1, (K \log t/t)^{1/3}\right\}$.
2. Choose the arm a_t:
   - If $u \leq \epsilon_t$ **then** choose $a_t \in \mathcal{A}$ uniformly at random (exploration).
   - Otherwise, $a_t \leftarrow \arg\max_{a \in \mathcal{A}} \bar{\mu}(a)$, breaking ties by the first arm index (exploitation).
3. Pull arm a_t and receive a reward r_t from the environment.
4. Update the pull count N_{a_t} and the sample mean $\bar{\mu}(a_t)$.

Optimism Under Uncertainty

Assume each arm is as good as it can be given the observations so far, and pull the arm with the best optimistic estimate.

The upper confidence bound of arm a at round t is $\bar{\mu}(a) + U(a, t)$, where the confidence radius is an allowance for how uncertain the sample mean still is:

$$U(a, t) = \sqrt{\frac{2 \log t}{N_a}},$$

and N_a is the number of times arm a has been pulled before round t.

With high probability, the upper confidence bound lies above the true mean $\mu(a)$. This idea leads to the UCB1 algorithm of Auer, Cesa-Bianchi, and Fischer (2002).

UCB1 Algorithm

Initialize: Given K arms and $T \geq K$ rounds, pull each arm once and call its reward r_a; set its pull count $N_a \leftarrow 1$ and sample mean $\bar{\mu}(a) \leftarrow r_a$.

For $t = K+1, \ldots, T$ **do**:

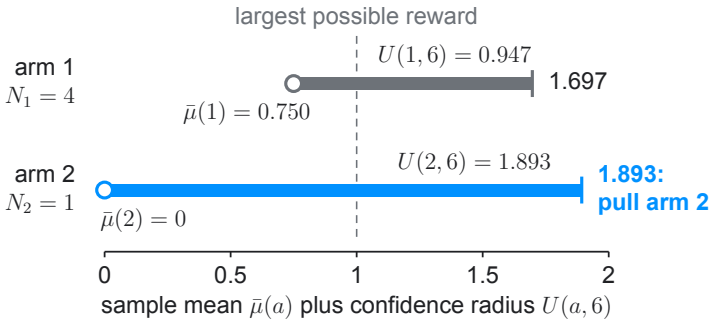
1. Compute the confidence radius $U(a,t)$ for each arm $a \in \mathcal{A}$.
2. Choose the arm with the largest upper confidence bound, $a_t \leftarrow \arg\max_{a \in \mathcal{A}} \{\bar{\mu}(a) + U(a,t)\}$, breaking ties by the first arm index.
3. Pull arm a_t and receive a reward r_t from the environment.
4. Update the pull count N_{a_t} and the sample mean $\bar{\mu}(a_t)$.

UCB1 achieves (Slivkins, Theorem 1.15):

$$\boxed{\mathbb{E}[R(T)] \leq \mathcal{O}\left(\sqrt{KT \log T}\right)}$$

Worked Example: Two Arms, Five Rounds

After five rounds, arm 1 has four pulls with $\bar{\mu}(1) = 0.75$, and arm 2 has one pull with $\bar{\mu}(2) = 0$. Before round $t = 6$:



One observation leaves more uncertainty than four. UCB1 pulls arm 2, whose bound 1.893 beats arm 1's 1.697, even above the largest possible reward of 1.

Comparing the Regret Bounds

Algorithm	How it explores	Expected-regret bound
Explore-first	pulls every arm N times, then commits to one arm	$T^{2/3} \cdot \mathcal{O}(K \log T)^{1/3}$
Epsilon-greedy	pulls a random arm with probability ϵ_t, which shrinks over time	$T^{2/3} \cdot \mathcal{O}(K \log T)^{1/3}$
UCB1	pulls the arm with the largest upper confidence bound	$\mathcal{O}\left(\sqrt{KT \log T}\right)$

All three grow more slowly than T, so the expected regret per round shrinks toward zero as the game gets longer. The UCB1 bound grows with about $\sqrt{T}$ rather than $T^{2/3}$, the strongest guarantee of the three.

Bounds from Slivkins, Theorems 1.5, 1.6, and 1.15. The epsilon-greedy bound, with the capped ϵ_t, holds at every round t.

Discuss

UCB1 has the strongest guarantee, yet explore-first is still common in practice. When might it be the better choice, for example when each pull is a slow experiment that we must plan in advance?

Binary Bernoulli Bandit Problem

Each reward is binary, $r_t \in \{0, 1\}$: arm a pays 1 with an unknown probability θ_a, so $r \sim \texttt{Bern}(\theta_a)$ and $\mu(a) = \theta_a$.

Ready to get your mind blown? We can represent our uncertainty about each θ_a with a prior, our belief before any rewards, and update it as rewards arrive:

- $\text{Beta}(\alpha, \beta)$ describes an unknown probability on $[0, 1]$; $\alpha, \beta > 0$ set its shape, its mean is $\alpha/(\alpha + \beta)$, and $\text{Beta}(1, 1)$ is uniform.
- After S successes and F failures, the posterior, our updated belief, is again a Beta distribution:

$$\boxed{\text{Beta}(\alpha, \beta) \;\longrightarrow\; \text{Beta}(\alpha + S,\, \beta + F)}$$

Epsilon-Greedy for the Bernoulli Bandit

The epsilon-greedy algorithm carries over with two changes:

1. Track the successes $S(a)$ and failures $F(a)$ of each arm, starting from $S(a) = F(a) = 0$ and uniform priors, $\alpha(a) = \beta(a) = 1$.
2. When exploiting, pull the arm with the highest posterior mean:

$$a_t \leftarrow \arg\max_{a \in \mathcal{A}} \left\{ \frac{\alpha(a) + S(a)}{\alpha(a) + \beta(a) + S(a) + F(a)} \right\}.$$

After each pull, update the counts of the arm we pulled:

$$S(a_t) \leftarrow S(a_t) + r_t, \qquad F(a_t) \leftarrow F(a_t) + (1 - r_t).$$

Everything else is unchanged: pull each arm once, then explore at random with probability ϵ_t.

Thompson Sampling Algorithm

Initialize: Given K arms and $T \geq 1$ rounds, set $\alpha(a) \leftarrow 1$, $\beta(a) \leftarrow 1$, $S(a) \leftarrow 0$, and $F(a) \leftarrow 0$ for each arm $a \in \mathcal{A}$.

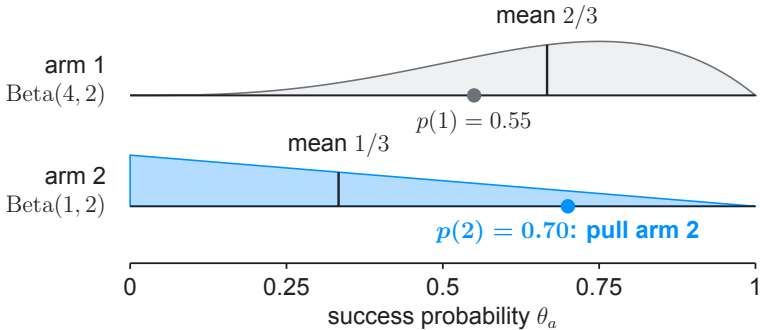
For $t = 1, 2, \ldots, T$ **do**:

1. Sample $p(a) \sim \text{Beta}\left(\alpha(a) + S(a),\ \beta(a) + F(a)\right)$ for every arm $a \in \mathcal{A}$.
2. Choose the arm with the highest sample, $a_t \leftarrow \arg\max_{a \in \mathcal{A}} \{p(a)\}$.
3. Pull arm a_t and receive $r_t \in \{0, 1\}$ from the environment.
4. Update $S(a_t) \leftarrow S(a_t) + r_t$ and $F(a_t) \leftarrow F(a_t) + (1 - r_t)$.

Thompson sampling explores mostly arms that could still be the best, so it often beats epsilon-greedy (Russo et al., 2020). Thompson (1933) proposed it for two treatments: give each one with the probability that it is the better one.

Worked Example: Updating a Bernoulli Arm

Both arms start from uniform $\mathrm{Beta}(1,1)$ priors. Arm 1 has 3 successes and 1 failure, and arm 2 has 1 failure.



Sampling gives an uncertain arm another trial. Arm 2's sample wins, so it is pulled; a success makes its posterior $\mathrm{Beta}(2,2)$, mean $1/2$.

Discuss

Suppose each arm is a set of reaction conditions and each pull is one experiment, a success if the yield clears a target. After a few weeks of experiments, what do the posteriors tell us that the success counts alone do not?

Lab: Bandit Algorithms

In the lab, we put these algorithms to work:

- we run explore-first, epsilon-greedy, and UCB1 on a bandit with seven arms, average their regret over 30 games, and compare how fast it grows with the bounds,
- we implement Thompson sampling for a binary Bernoulli bandit and watch the posterior of the best arm concentrate around its true success probability, while the arms it rarely pulls stay uncertain.

Lab: Bandit algorithms

Summary

We introduced the stochastic multi-armed bandit problem and four algorithms for balancing exploration and exploitation.

Key takeaways

- **Exploration and exploitation.** Too little exploration can lock an agent onto an arm that only looked good by chance.
- **Bandit algorithms.** The four algorithms explore up front, at random, optimistically, or by sampling from posteriors.
- **Regret bounds.** The UCB1 bound grows with about $\sqrt{T}$, more slowly than the $T^{2/3}$ bounds of explore-first and epsilon-greedy.

When actions also change the state of the environment, we use a Markov decision process.